

UNIVERSIDAD TECNOLÓGICA FIDEL VELÁZQUEZ

DIVISIÓN ACADÉMICA DE INFORMÁTICA

“KASA SERVICE TRACKER”

Sistema de seguimiento de órdenes de servicio automotriz

Proyecto académico de Ingeniería de Software

Lenguaje principal: C#

Plataforma: .NET

Metodología de desarrollo: Kanban

Modalidad: Desarrollo individual

Contexto de aplicación: Grupo Kasa Automotriz

P R E S E N T A

Rivera Delgado Luis Enrique

TLALNEPANTLA ESTADO DE MÉXICO,

MAYO 2024

Índice

1. INTRODUCCIÓN	4
2. PLANTEAMIENTO DEL PROBLEMA	5
3. JUSTIFICACIÓN.....	6
4. OBJETIVO GENERAL.....	7
5. OBJETIVOS ESPECÍFICOS.....	7
6. ALCANCE DEL PROYECTO	7
7. DESCRIPCIÓN GENERAL DEL SISTEMA	8
8. FLUJO DE ESTADOS	9
9. PERSISTENCIA Y ALMACENAMIENTO DE INFORMACIÓN	10
10. ORGANIZACIÓN DE LOS REGISTROS	11
11. CONSULTA DEL HISTORIAL	12
12. EDICIÓN Y CORRECCIÓN DE REGISTROS	12
13. GENERACIÓN DE TICKETS	13
14. ARQUITECTURA DEL SOFTWARE	14
15. PATRONES Y PRINCIPIOS UTILIZADOS	14
16. VALIDACIÓN DE DATOS.....	15
17. MANEJO DE ERRORES.....	16
18. PRUEBAS DE SOFTWARE	16
19. TRAZABILIDAD	17
20. SEGURIDAD E INTEGRIDAD DE LA INFORMACIÓN.....	17
21. METODOLOGÍA KANBAN.....	18
21.1 Estructura del tablero.....	19
21.2 Gestión del trabajo en progreso.....	20
21.3 Evolución incremental del sistema	20
21.4 Clasificación de actividades.....	21
21.5 Gestión de funcionalidades.....	22
21.6 Incorporación de mejoras detectadas durante el desarrollo.....	22
21.7 Gestión de incidencias	23
21.8 Definition of Ready.....	24
21.9 Definition of Done.....	24
21.10 Etapa final del tablero	25
21.11 Resultados de la aplicación de Kanban	26

22. CONTROL DE VERSIONES	26
23. INSTALACIÓN Y EJECUCIÓN.....	27
24. ESTRUCTURA DE DIRECTORIOS	27
25. RESPALDO Y RECUPERACIÓN.....	28
26. MANTENIMIENTO	28
27. LIMITACIONES ACTUALES.....	29
28. RESULTADOS DEL PROYECTO	30
29. CONCLUSIONES	31
30. COMPETENCIAS TÉCNICAS DESARROLLADAS	32
31. UML	32
UML 1 — Diagrama de casos de uso	32
UML 2 — Diagrama de clases	33
UML 3 — Diagrama de secuencia para registrar una orden	34
UML 4 — Diagrama de secuencia para entrega y finalización	34
UML 5 — Diagrama de actividad	35
UML 6 — Diagrama de componentes.....	36
UML 7 — Diagrama de despliegue.....	36
32. ANEXOS	37

1. INTRODUCCIÓN

KASA Service Tracker es un proyecto académico de desarrollo de software orientado a la administración y seguimiento de órdenes de servicio dentro de un entorno automotriz. El proyecto fue planteado con el propósito de aplicar conocimientos de programación en C#, programación orientada a objetos, persistencia de información, control de estados, manejo de archivos, pruebas de software y organización de un proyecto mediante una metodología ágil.

La aplicación busca representar de manera simplificada el flujo que puede existir desde el momento en que un vehículo es recibido para servicio hasta que la atención correspondiente es terminada y el vehículo es entregado al usuario. Para ello, el sistema permite registrar información asociada al automóvil y al servicio solicitado, consultar órdenes previamente registradas, realizar correcciones sobre los datos capturados, actualizar el estado de la atención, mantener un historial de modificaciones y generar comprobantes durante la entrega.

Uno de los objetivos principales del desarrollo fue conservar un alcance compacto y realista para un proyecto elaborado de manera individual. En lugar de construir un sistema empresarial de gran tamaño con módulos de facturación, inventarios, recursos humanos o múltiples sucursales, se decidió concentrar el desarrollo en una función específica: el seguimiento estructurado de órdenes de servicio automotriz.

Esta decisión permitió dedicar mayor atención a la organización interna del código, la validación de datos, la persistencia de información, el manejo de errores y la creación de un flujo de trabajo coherente. De esta manera, el proyecto no se limita únicamente a demostrar conocimientos básicos de sintaxis en C#, sino que representa una aplicación completa con diferentes componentes relacionados entre sí.

El uso del nombre Grupo Kasa Automotriz corresponde exclusivamente al contexto académico autorizado para el desarrollo del proyecto. KASA Service

Tracker no representa un sistema oficial de producción de dicha organización ni pretende sustituir alguno de sus sistemas internos.

2. PLANTEAMIENTO DEL PROBLEMA

Dentro de un entorno de servicio automotriz se genera continuamente información relacionada con vehículos que ingresan para recibir mantenimiento, revisión o reparación. Cuando el volumen de unidades aumenta, administrar esta información únicamente mediante registros manuales o listas extensas puede dificultar la localización de una orden específica, provocar errores de captura y complicar el seguimiento del estado actual de cada automóvil.

Una de las principales dificultades de utilizar un registro acumulativo consiste en que las órdenes permanecen visibles aun cuando los servicios correspondientes ya fueron concluidos. Conforme transcurren las semanas, la cantidad de registros disponibles aumenta y la consulta cotidiana pierde eficiencia. Aunque un sistema informático puede procesar cientos o miles de objetos sin problemas graves de rendimiento, presentar toda esa información simultáneamente al operador no representa una buena experiencia de uso.

Además, durante la captura de información pueden producirse equivocaciones relacionadas con placas, modelos, descripción del servicio, año del automóvil o importes estimados. Un sistema que no permita corregir estos datos obligaría a eliminar registros o crear órdenes nuevas, ocasionando inconsistencias en la información.

También resulta necesario conservar evidencia histórica sobre el recorrido de una orden. No solamente interesa conocer cuál es el estado actual del vehículo, sino cuándo fue recibido, cuándo pasó a diagnóstico, cuándo estuvo en reparación, cuándo fue entregado y si se realizaron correcciones durante el proceso.

A partir de estas necesidades se planteó KASA Service Tracker como una herramienta compacta de seguimiento. La aplicación mantiene visibles

principalmente las órdenes que continúan siendo relevantes para la operación reciente, mientras que la información histórica se conserva mediante almacenamiento local y archivos de respaldo organizados por mes y año.

3. JUSTIFICACIÓN

El desarrollo de KASA Service Tracker permite abordar un problema operativo concreto mediante una solución de software pequeña pero estructurada. Esta característica resulta importante desde el punto de vista académico, ya que permite aplicar conceptos de ingeniería de software sin depender de un proyecto excesivamente grande cuya complejidad dificulte completar todas sus etapas.

El proyecto fue desarrollado utilizando C# y .NET debido a que estas tecnologías permiten construir aplicaciones robustas bajo un paradigma orientado a objetos y ofrecen herramientas adecuadas para el manejo de archivos, colecciones, validaciones, serialización de información y pruebas automatizadas.

La aplicación también incorpora elementos relacionados con el ciclo de vida de la información. Una orden no desaparece simplemente cuando el vehículo es entregado. Por el contrario, la información correspondiente permanece disponible para búsquedas posteriores y es respaldada en archivos históricos. Esto permite representar un comportamiento más cercano a una aplicación utilizada dentro de un ambiente administrativo real.

Asimismo, el proyecto constituye una oportunidad para demostrar competencias relacionadas con organización de código, separación de responsabilidades, tratamiento de errores, construcción de pruebas unitarias y aplicación de una metodología de trabajo. Estas competencias complementan el desarrollo puramente funcional del programa y permiten presentar KASA Service Tracker como evidencia de conocimientos de ingeniería de software.

4. OBJETIVO GENERAL

Desarrollar una aplicación en C# capaz de registrar, consultar y dar seguimiento a órdenes de servicio automotriz, manteniendo trazabilidad sobre los cambios realizados y conservando la información histórica mediante mecanismos locales de persistencia y exportación.

5. OBJETIVOS ESPECÍFICOS

El sistema tiene como propósito registrar los datos principales de cada vehículo y del servicio solicitado, generar un folio que permita identificar individualmente cada orden y mantener un flujo organizado de estados durante el desarrollo del servicio.

De igual manera, se busca facilitar la localización de información mediante consultas por folio y placas, evitando depender exclusivamente de listas generales de registros. Se incorpora también un mecanismo de corrección para solucionar errores de captura sin necesidad de eliminar la orden original.

Otro de los objetivos consiste en conservar un historial de cambios que permita observar la evolución de una orden durante todo su ciclo de vida. Finalmente, se pretende generar comprobantes para la empresa y el usuario durante la entrega, así como respaldar automáticamente la información correspondiente en archivos organizados por periodos mensuales.

6. ALCANCE DEL PROYECTO

KASA Service Tracker fue diseñado como un sistema local de seguimiento de servicios automotrices. Su alcance comprende el registro de vehículos, generación de órdenes de servicio, consultas, modificaciones controladas, seguimiento de estados, generación de tickets y almacenamiento histórico.

El sistema administra información relacionada con las placas del vehículo, marca, modelo, año, descripción del servicio, costo estimado, fecha de creación, estado de la orden y modificaciones realizadas durante su procesamiento.

La aplicación permite mostrar las órdenes correspondientes al periodo operativo actual sin necesidad de presentar continuamente todo el histórico. Cuando una orden concluye correctamente, puede dejar de formar parte del listado semanal sin que esto signifique eliminar su información.

Los registros continúan almacenados dentro de la persistencia interna de la aplicación y son además respaldados mediante archivos de Excel organizados automáticamente utilizando el mes y año correspondientes.

El sistema fue desarrollado como una aplicación local para Windows. No contempla conexión con servidores externos, servicios en la nube ni plataformas corporativas.

Quedan fuera del alcance funciones como facturación electrónica, administración completa de inventarios, pagos, administración de personal, autenticación empresarial, comunicación con sucursales, seguimiento GPS, aplicaciones móviles o integración con infraestructura oficial de Grupo Kasa Automotriz.

7. DESCRIPCIÓN GENERAL DEL SISTEMA

El funcionamiento de KASA Service Tracker comienza con el registro de una nueva orden. El operador introduce los datos correspondientes al vehículo y al servicio solicitado. Antes de almacenar la información, los valores de texto son normalizados automáticamente mediante conversión a mayúsculas.

Esta característica evita inconsistencias visuales provocadas por capturas como “Nissan”, “nissan” o “NISSAN”, haciendo que todos los registros mantengan el mismo formato independientemente de la forma en que el usuario los introduzca.

Después del registro, cada orden puede ser consultada mediante su folio o mediante las placas del automóvil. El sistema también proporciona una opción para mostrar las órdenes relacionadas con el periodo semanal operativo.

A diferencia de una lista histórica permanente, este mecanismo permite trabajar principalmente con las órdenes recientes y activas. Los servicios previamente terminados pueden permanecer archivados sin saturar la vista principal.

Cuando se detecta un error de captura, el operador puede utilizar la función de edición. Esta función permite modificar únicamente aquellos datos que tienen sentido desde el punto de vista administrativo, como placas, marca, modelo, año, descripción del servicio y costo estimado.

El folio de la orden no puede ser modificado porque constituye el identificador principal del registro. Tampoco se permite alterar directamente el estado desde la función de edición, ya que esta operación dispone de un flujo específico dentro del sistema.

Durante el desarrollo del servicio, la orden avanza entre distintos estados. Cada modificación queda asociada al historial, permitiendo posteriormente conocer la evolución del servicio.

Cuando el vehículo alcanza el proceso de entrega, el sistema genera dos comprobantes: uno destinado a la empresa y otro destinado al usuario. Después de completar correctamente esta operación, la orden puede cambiar automáticamente a su estado final.

La información continúa disponible para futuras consultas, reimpresión de tickets o revisión del historial.

8. FLUJO DE ESTADOS

El flujo principal de una orden fue diseñado para representar de manera simplificada el recorrido de un automóvil dentro del servicio.

Una orden comienza en estado **RECIBIDO**, indicando que los datos iniciales del vehículo y del servicio fueron registrados correctamente.

Posteriormente puede avanzar a **DIAGNÓSTICO**, estado utilizado para representar el análisis inicial del automóvil y la identificación de las actividades necesarias.

Después continúa como **EN REPARACIÓN**, indicando que la actividad correspondiente se encuentra siendo ejecutada.

Una vez que el servicio ha concluido y el vehículo está preparado para regresar al usuario, la orden cambia a **ENTREGADO**. Durante esta transición se genera el proceso de impresión de comprobantes para empresa y usuario.

Después de la correcta generación del ticket, el sistema establece la orden como **TERMINADO**, representando el cierre definitivo del servicio dentro del flujo operativo.

El objetivo de separar las transiciones consiste en evitar modificaciones arbitrarias que puedan producir inconsistencias dentro del historial.

9. PERSISTENCIA Y ALMACENAMIENTO DE INFORMACIÓN

KASA Service Tracker utiliza archivos JSON como mecanismo de persistencia operacional. Esto significa que los objetos gestionados por la aplicación pueden almacenarse en disco y posteriormente recuperarse cuando el programa vuelve a ejecutarse.

La utilización de JSON resulta adecuada para el alcance del proyecto debido a que permite conservar estructuras de datos de manera legible y sencilla sin necesidad de instalar o administrar un servidor de bases de datos.

La información histórica también es exportada a archivos de Excel organizados utilizando el mes y año correspondientes. Estos documentos tienen como propósito facilitar la consulta administrativa y mantener un respaldo estructurado de las órdenes registradas.

Excel no funciona como motor principal de persistencia de la aplicación. La decisión de mantener JSON como almacenamiento operacional evita depender de un documento que puede encontrarse abierto por otro usuario, ser modificado manualmente o presentar bloqueos mientras se ejecuta el programa.

De esta manera se establece una separación entre el almacenamiento utilizado internamente por la aplicación y los documentos destinados a consulta o respaldo.

10. ORGANIZACIÓN DE LOS REGISTROS

Una de las decisiones funcionales más importantes consistió en evitar que la opción destinada a listar órdenes se convirtiera en una salida acumulativa indefinida.

En un escenario real es posible registrar una cantidad considerable de vehículos durante una semana. Mostrar continuamente todos los servicios realizados durante meses anteriores no aportaría utilidad al trabajo cotidiano del operador y dificultaría localizar información reciente.

Por este motivo, el listado se enfoca en el periodo semanal operativo. Las órdenes activas continúan siendo visibles mientras se encuentran dentro del flujo de servicio. Aquellas que alcanzan correctamente el estado final pueden dejar de mostrarse en la lista principal.

Esta decisión no implica eliminar información. Los registros terminados permanecen disponibles a través de las funciones de búsqueda, historial y respaldo mensual en Excel.

Con esta estrategia se diferencia claramente entre información operativa e información histórica.

11. CONSULTA DEL HISTORIAL

La función de historial fue desarrollada para consultar específicamente la evolución de una orden individual.

El operador proporciona el folio de la orden y el sistema recupera los eventos asociados. De esta manera es posible conocer los estados por los que atravesó el servicio y cualquier modificación relevante realizada durante su existencia.

El historial representa una característica de trazabilidad. Su propósito no es funcionar como una lista general de todos los vehículos, sino responder preguntas relacionadas con un servicio determinado.

Por ejemplo, si un usuario solicita información relacionada con un automóvil atendido varios días anteriormente, el operador puede localizar la orden por su folio o placas y posteriormente consultar su historial sin necesidad de revisar manualmente todos los servicios realizados durante ese periodo.

12. EDICIÓN Y CORRECCIÓN DE REGISTROS

La aplicación incluye una función específica para corregir información previamente registrada. Esta funcionalidad responde a la posibilidad de que durante la captura inicial puedan existir errores humanos.

Al ingresar a la edición, el operador identifica la orden mediante su folio y posteriormente puede modificar únicamente los campos autorizados.

Los valores que no requieren modificación pueden conservarse sin necesidad de volver a capturarlos.

El sistema impide utilizar este procedimiento para modificar elementos críticos como el identificador principal de la orden o el flujo de estados.

Esta separación ayuda a mantener la integridad de la información y evita utilizar una herramienta administrativa de edición como mecanismo para alterar la lógica del servicio.

Todas las modificaciones textuales continúan pasando por el proceso automático de normalización a mayúsculas.

13. GENERACIÓN DE TICKETS

La etapa de entrega incluye la generación de dos comprobantes. El primero corresponde a la copia destinada a la empresa y el segundo corresponde a la copia destinada al usuario.

Los tickets contienen información que permite identificar la orden, el vehículo y el servicio relacionado. Además de ser enviados al proceso de impresión, los archivos pueden conservarse localmente para disponer de evidencia ante cualquier problema ocurrido durante la impresión física.

La aplicación incluye también una función de consulta y reimpresión. Esta opción permite recuperar la información asociada a un ticket previamente generado y preguntar al operador si desea producir nuevamente las copias.

El mecanismo de reimpresión evita alterar innecesariamente el estado de la orden, ya que su objetivo únicamente consiste en recuperar un comprobante asociado a un servicio terminado.

14. ARQUITECTURA DEL SOFTWARE

El proyecto utiliza una organización basada en separación de responsabilidades.

Las clases pertenecientes al dominio representan los principales elementos de información administrados por la aplicación, como vehículos, órdenes de servicio, estados e historial.

La lógica relacionada con las operaciones del sistema se concentra en servicios. De esta manera, la interfaz de consola no necesita encargarse directamente de todas las reglas de negocio.

Por otra parte, el acceso a la persistencia se encuentra separado mediante repositorios. Esta organización permite que el código encargado de guardar y recuperar información no se mezcle completamente con las operaciones de presentación o validación.

Aunque KASA Service Tracker es un proyecto pequeño, esta separación fue implementada con el objetivo de mantener una estructura escalable y comprensible.

La arquitectura puede representarse conceptualmente mediante cuatro niveles: interfaz, servicios, dominio y persistencia.

La interfaz contiene la interacción con el usuario. La capa de servicios procesa las operaciones solicitadas y aplica reglas del sistema. El dominio representa los objetos utilizados por la aplicación y la persistencia se encarga de conservar o recuperar la información.

15. PATRONES Y PRINCIPIOS UTILIZADOS

Dentro del proyecto se utiliza una implementación sencilla del patrón Repository para desacoplar el acceso a archivos del resto de la lógica.

El servicio principal depende de una abstracción de repositorio y no necesariamente de una implementación específica. Este planteamiento permite sustituir en el futuro el almacenamiento basado en JSON por otro mecanismo sin tener que reescribir completamente la lógica del programa.

También se aplica separación de responsabilidades. Las clases encargadas de representar información no realizan las mismas funciones que aquellas encargadas de interactuar con el usuario o guardar archivos.

La aplicación utiliza programación orientada a objetos para encapsular propiedades y comportamientos relacionados con las entidades administradas.

Asimismo, se utiliza LINQ para realizar operaciones de consulta sobre colecciones de órdenes, facilitando búsquedas y filtros sin necesidad de implementar manualmente todos los ciclos de recorrido.

16. VALIDACIÓN DE DATOS

La validación es necesaria para evitar que información incorrecta llegue al almacenamiento.

El sistema controla campos obligatorios y valores numéricos utilizados en elementos como año o costo estimado.

También se implementan validaciones para impedir estados o modificaciones que puedan romper el flujo normal de una orden.

Los textos registrados son normalizados a mayúsculas antes de almacenarse. Esta característica representa una regla del sistema y no únicamente una característica visual de la consola.

Por esa razón, la transformación se aplica dentro de la lógica de negocio para asegurar que los datos mantengan el formato incluso si en el futuro se utiliza otra interfaz para trabajar con el sistema.

17. MANEJO DE ERRORES

El proyecto contempla la posibilidad de recibir información inválida o encontrar problemas relacionados con archivos.

Para estos casos se utilizan mecanismos de validación y manejo de excepciones. La finalidad es evitar que una entrada incorrecta provoque directamente la terminación inesperada de la aplicación.

Los errores deben presentarse mediante mensajes comprensibles que permitan al operador corregir la acción realizada.

El manejo adecuado de errores también ayuda a separar aquellos problemas que corresponden a una captura incorrecta de aquellos relacionados con almacenamiento o funcionamiento interno.

18. PRUEBAS DE SOFTWARE

El proyecto incorpora pruebas automatizadas mediante xUnit para validar comportamientos importantes de la lógica.

Durante el desarrollo se verificaron operaciones relacionadas con registro de órdenes, consultas, cambios de estado, historial, validaciones y edición de información.

Uno de los objetivos de las pruebas consiste en confirmar que una modificación realizada posteriormente no altere accidentalmente funcionalidades que previamente funcionaban.

Las pruebas automatizadas también permitieron detectar problemas durante la preparación inicial del proyecto. Por ejemplo, durante una de las primeras compilaciones el proyecto de pruebas no reconocía los atributos utilizados por xUnit debido a la ausencia de la directiva correspondiente dentro del archivo de pruebas.

El problema fue identificado mediante los mensajes proporcionados por el compilador, corregido y posteriormente validado mediante una nueva ejecución.

Este tipo de incidencias forma parte normal del proceso de desarrollo y representa evidencia de actividades relacionadas con diagnóstico y resolución de problemas.

Además de las pruebas automatizadas, se realizaron pruebas manuales ejecutando el flujo completo desde la consola.

Estas pruebas incluyeron registro de un vehículo, consulta de la orden generada, búsqueda por placas, cambio de estados, visualización del historial y verificación de persistencia después de cerrar y ejecutar nuevamente el sistema.

19. TRAZABILIDAD

La trazabilidad constituye uno de los elementos principales del proyecto.

Cada orden cuenta con un identificador que permite localizarla independientemente de su posición dentro de la lista semanal.

El historial conserva información relacionada con la evolución de la orden, mientras que los respaldos mensuales permiten mantener evidencia histórica incluso después de que el servicio dejó de formar parte del trabajo cotidiano.

La combinación de folio, historial, persistencia JSON, tickets y exportación mensual permite evitar que el cierre de una orden represente la pérdida de información.

20. SEGURIDAD E INTEGRIDAD DE LA INFORMACIÓN

Debido a que KASA Service Tracker es una aplicación local y no se encuentra expuesta directamente a Internet, las necesidades de seguridad son diferentes a las de una aplicación web.

La principal prioridad consiste en proteger la integridad de la información.

Para ello, el sistema restringe la modificación de identificadores, controla el cambio de estados, valida datos de entrada y mantiene respaldos históricos.

El proyecto no implementa autenticación de usuarios ni permisos por roles debido a que estas características se encuentran fuera del alcance definido.

Si el proyecto evolucionara hacia una solución empresarial con múltiples operadores, sería necesario incorporar autenticación, autorización, auditoría individual de acciones y almacenamiento mediante un sistema gestor de bases de datos.

21. METODOLOGÍA KANBAN

Para la organización y seguimiento del desarrollo de KASA Service Tracker se utilizó la metodología Kanban, implementada mediante la plataforma Trello. Su aplicación permitió representar de forma visual las actividades realizadas durante el proyecto, controlar el trabajo en progreso y mantener una secuencia clara desde la identificación de una necesidad hasta su implementación, prueba y finalización.

Debido a que el proyecto fue desarrollado de manera individual, la metodología fue adaptada a un flujo de trabajo para un solo desarrollador. No se buscó reproducir artificialmente la estructura de un equipo de desarrollo de gran tamaño, sino aplicar los principios de Kanban de manera proporcional al alcance del sistema.

El tablero fue denominado **“KASA Service Tracker — Desarrollo Kanban”** y se estructuró utilizando las columnas **Backlog, Por hacer, En progreso, Pruebas, Revisión y Terminado**. Cada tarjeta representa una actividad funcional, técnica, de calidad, persistencia o despliegue realizada durante el desarrollo.

Esta organización permitió visualizar de manera inmediata qué actividades se encontraban pendientes, cuáles estaban siendo desarrolladas, cuáles requerían validación y cuáles habían cumplido completamente con los criterios definidos para considerarse terminadas.

21.1 Estructura del tablero

La primera columna, **Backlog**, fue utilizada para concentrar las funcionalidades o mejoras identificadas que todavía no se encontraban preparadas para iniciar su implementación.

Cuando una actividad ya tenía claramente definido su propósito, alcance y resultado esperado, podía trasladarse a **Por hacer**. Esta columna representaba las tareas listas para ser atendidas en cuanto existiera capacidad disponible.

La columna **En progreso** correspondió al desarrollo activo. Debido a que KASA Service Tracker fue realizado por un único desarrollador, se estableció como política mantener únicamente una actividad principal en esta etapa. Esta restricción permitió evitar la ejecución simultánea de múltiples funcionalidades y favoreció la terminación de cada elemento antes de comenzar otro.

Después de completar la implementación, la tarjeta avanzaba hacia **Pruebas**. En esta etapa se comprobaba que el comportamiento desarrollado funcionara correctamente tanto de manera individual como integrado con las funciones existentes.

Posteriormente, la actividad pasaba a **Revisión**, etapa utilizada para comprobar aspectos relacionados con estructura del código, validaciones, comportamiento general, persistencia de información y posibles efectos sobre funcionalidades previamente terminadas.

Finalmente, una tarjeta podía trasladarse a **Terminado** únicamente cuando la funcionalidad había sido implementada, probada y considerada estable dentro del alcance establecido.

21.2 Gestión del trabajo en progreso

Uno de los principios aplicados durante el desarrollo fue la limitación del trabajo en progreso o WIP.

Al tratarse de un proyecto individual, mantener múltiples funcionalidades desarrollándose simultáneamente hubiera aumentado la posibilidad de dejar actividades incompletas o introducir errores difíciles de identificar.

Por este motivo, se estableció un límite de una tarjeta principal en la columna **En progreso** y una actividad en **Pruebas**. De esta manera, una nueva funcionalidad no debía comenzar mientras existiera otra que todavía requiriera terminar su implementación o validación.

Esta política resultó especialmente útil debido a que varias mejoras surgieron después de utilizar las funciones previamente desarrolladas. En lugar de intentar anticipar desde el inicio todas las características del sistema, el proyecto evolucionó mediante pequeñas entregas funcionales.

21.3 Evolución incremental del sistema

KASA Service Tracker no fue desarrollado como una única implementación extensa. El sistema evolucionó de manera incremental conforme se completaban y evaluaban funcionalidades individuales.

Las primeras actividades estuvieron relacionadas con la estructura del proyecto, registro de órdenes, persistencia mediante JSON, consultas y flujo de estados.

Una vez que estas funciones se encontraban operativas, se incorporaron características relacionadas con historial, generación de tickets y finalización automática de órdenes.

Durante las pruebas también se identificaron nuevas necesidades que originalmente no formaban parte del funcionamiento inicial. Entre ellas se encontró

la posibilidad de corregir datos previamente registrados, la normalización automática de información a mayúsculas y la reorganización del listado de órdenes.

Este último cambio surgió al analizar el comportamiento que tendría el sistema después de registrar una gran cantidad de servicios. Mantener todas las órdenes históricas visibles dentro del listado principal hubiera reducido considerablemente la utilidad de dicha función. Como resultado, se desarrolló un listado operativo basado en el periodo semanal, mientras que los registros terminados continuaron disponibles mediante búsquedas, historial y archivos históricos.

Posteriormente se incorporó la exportación automática de registros a archivos Excel organizados por mes y año, así como el archivado de órdenes terminadas.

El tablero Kanban permitió registrar cada una de estas mejoras como actividades independientes, mostrando claramente la evolución del sistema desde su primera versión funcional hasta la versión final.

21.4 Clasificación de actividades

Para facilitar la identificación de los distintos tipos de trabajo se utilizaron etiquetas dentro del tablero Trello.

Las actividades relacionadas directamente con funciones utilizadas por el operador fueron identificadas como **Funcionalidad**. Las tareas relacionadas con almacenamiento JSON, archivos históricos y exportación a Excel fueron clasificadas dentro de **Datos/Persistencia**.

Las actividades destinadas a pruebas, validaciones y solución de errores se identificaron mediante la categoría **Calidad/Testing**, mientras que aquellas relacionadas con instalación, ejecución y acceso directo fueron clasificadas como **Despliegue**.

Finalmente, se utilizó una categoría denominada **Mejora/Refactor** para representar modificaciones que surgieron después de evaluar funcionalidades previamente implementadas.

La utilización de estas categorías permitió observar que el desarrollo no estuvo concentrado exclusivamente en programación funcional, sino también en calidad, almacenamiento, mantenimiento y preparación del producto para su utilización.

21.5 Gestión de funcionalidades

Cada tarjeta del tablero fue utilizada para representar una unidad concreta de trabajo.

Las actividades principales incluyeron la creación de la estructura del proyecto, registro de órdenes, persistencia JSON, búsqueda por folio y placas, implementación del flujo de estados, historial, generación de tickets, reimpresión, edición de registros, normalización de información, listado semanal, exportación mensual a Excel, archivado y preparación del acceso directo para Windows.

Las tarjetas que requerían varios pasos fueron divididas mediante listas de comprobación. Por ejemplo, el registro de una orden implicaba capturar información del vehículo, validar datos, generar el folio correspondiente, asignar un estado inicial y conservar la información mediante el mecanismo de persistencia.

Esta división permitió comprobar el progreso interno de una actividad sin tener que fragmentar excesivamente el tablero en tareas demasiado pequeñas.

21.6 Incorporación de mejoras detectadas durante el desarrollo

Una característica importante de la aplicación de Kanban fue la posibilidad de incorporar nuevas necesidades sin alterar completamente la planificación del proyecto.

Por ejemplo, después de implementar el listado general de órdenes se identificó que esta función podría perder utilidad conforme aumentara la cantidad de vehículos registrados.

A partir de este análisis se creó una nueva actividad denominada **Optimizar listado mediante periodo semanal**. Esta mejora tuvo como finalidad conservar en la vista cotidiana únicamente la información necesaria para la operación reciente, mientras que las órdenes finalizadas permanecieran disponibles dentro del almacenamiento histórico.

Una situación similar ocurrió con la edición de registros. Durante la evaluación del flujo se consideró la posibilidad de que un operador capturara incorrectamente una placa, modelo, año o descripción del servicio. En lugar de eliminar completamente una orden, se incorporó una función específica para corregir información manteniendo la trazabilidad correspondiente.

La generación de tickets también evolucionó durante el desarrollo. Inicialmente se consideró únicamente un comprobante de entrega y posteriormente se estableció la generación de dos copias independientes, una para la empresa y otra para el usuario, incluyendo además la posibilidad de reimpresión.

Estos cambios muestran el uso de Kanban como herramienta de adaptación y no únicamente como mecanismo visual para clasificar tareas.

21.7 Gestión de incidencias

El tablero también permitió documentar problemas detectados durante el proceso de desarrollo.

Durante una de las primeras compilaciones, el proyecto principal fue construido correctamente; sin embargo, el proyecto de pruebas presentó errores debido a que los atributos utilizados por xUnit no eran reconocidos por el compilador.

El análisis del mensaje de error permitió identificar que el archivo correspondiente a las pruebas no incluía la directiva necesaria para utilizar xUnit.

Una vez incorporada la referencia, el proyecto volvió a compilarse y las pruebas pudieron ejecutarse correctamente.

Esta incidencia fue registrada como actividad de calidad debido a que su solución formó parte del proceso real de construcción y validación del software.

Documentar este tipo de situaciones permite mostrar que el desarrollo no consistió únicamente en implementar funcionalidades, sino también en diagnosticar errores y aplicar correcciones verificables.

21.8 Definition of Ready

Para evitar iniciar actividades insuficientemente definidas, se utilizó un criterio de preparación antes de trasladar una tarjeta hacia el trabajo activo.

Una actividad debía contar con un objetivo entendible, un resultado esperado y suficiente información para comenzar su implementación.

También debía mantenerse dentro del alcance establecido para KASA Service Tracker y ser lo suficientemente pequeña para completarse de forma individual sin generar dependencias innecesarias.

La aplicación de este criterio permitió reducir cambios de dirección durante la implementación y mantener un flujo de trabajo estable.

21.9 Definition of Done

Una actividad no se consideró terminada únicamente porque el código correspondiente hubiera sido escrito.

Para trasladar una tarjeta a la columna **Terminado**, la funcionalidad debía encontrarse implementada, compilar correctamente y cumplir con el comportamiento esperado.

Cuando correspondía, también debía comprobarse la persistencia de información, validación de entradas y funcionamiento junto con el resto del sistema.

En las funciones críticas se realizaron pruebas adicionales antes de su cierre.

Esta definición permitió distinguir claramente entre una actividad técnicamente desarrollada y una actividad realmente terminada dentro del proyecto.

21.10 Etapa final del tablero

Después de completar las funcionalidades individuales se creó una actividad denominada **Validación final versión 1.0**.

Esta tarjeta representa el cierre global del desarrollo y no únicamente una función adicional.

Durante esta etapa se realizó la comprobación conjunta de compilación, ejecución, persistencia, consultas, edición, cambio de estados, historial, tickets, exportación a Excel y funcionamiento general de la aplicación.

La tarjeta avanzó a través del mismo flujo utilizado durante el resto del proyecto: Por hacer, En progreso, Pruebas, Revisión y finalmente Terminado.

Este procedimiento permitió utilizar el tablero no solamente para controlar funcionalidades individuales, sino también para representar formalmente el cierre de la primera versión estable del sistema.

21.11 Resultados de la aplicación de Kanban

La utilización de Kanban permitió mantener el proyecto dentro de un alcance controlado y facilitar la incorporación de mejoras conforme se evaluaba el funcionamiento real del sistema.

La visualización del trabajo permitió diferenciar claramente entre actividades pendientes, desarrollo activo, validación y tareas terminadas.

La limitación del trabajo en progreso resultó adecuada para un proyecto individual, ya que favoreció la terminación de una funcionalidad antes de iniciar otra.

También permitió representar de manera explícita tareas relacionadas con pruebas, almacenamiento y despliegue, evitando considerar que el desarrollo de software consiste únicamente en escribir código.

El resultado final fue un flujo de trabajo incremental en el que KASA Service Tracker evolucionó desde una estructura inicial hasta una versión estable con registro de órdenes, persistencia, consultas, seguimiento, correcciones, tickets, almacenamiento histórico y herramientas de ejecución.

La evidencia obtenida mediante el tablero de Trello forma parte de la documentación técnica del proyecto y permite observar gráficamente la aplicación de la metodología durante su desarrollo.

22. CONTROL DE VERSIONES

El desarrollo puede mantenerse mediante Git y GitHub para registrar los cambios realizados en el código y la documentación.

El control de versiones permite conservar un historial técnico del proyecto y facilita identificar cuándo fue incorporada cada funcionalidad.

Las versiones principales del sistema pueden organizarse utilizando versionado semántico. Bajo este esquema, una versión estable del proyecto puede identificarse como 1.0.0, mientras que futuras correcciones o mejoras pueden modificar los valores correspondientes a versión mayor, menor o parche.

El repositorio puede incluir ramas independientes para nuevas funcionalidades antes de integrarlas a la rama principal.

Aunque el proyecto fue desarrollado individualmente, utilizar este procedimiento ayuda a demostrar conocimiento sobre prácticas utilizadas dentro de equipos de desarrollo colaborativos.

23. INSTALACIÓN Y EJECUCIÓN

KASA Service Tracker se encuentra diseñado para ejecutarse en Windows.

Durante el entorno de desarrollo, la aplicación puede compilarse utilizando el SDK de .NET.

Una vez restauradas las dependencias del proyecto, la compilación permite comprobar que tanto la aplicación principal como el proyecto de pruebas se encuentran correctamente configurados.

La estructura final incluye también un archivo BAT preparado para facilitar la creación de un acceso directo en el escritorio.

Esto evita que un usuario final tenga que navegar manualmente por las carpetas del proyecto o introducir comandos técnicos cada vez que desea utilizar el programa.

24. ESTRUCTURA DE DIRECTORIOS

La estructura de archivos del proyecto fue organizada separando código fuente, pruebas, datos y archivos generados.

El código principal se encuentra dentro de la carpeta correspondiente a la aplicación, mientras que las pruebas se mantienen en un proyecto independiente.

Los archivos utilizados como persistencia se encuentran separados de los documentos de respaldo mensual y de los tickets generados.

Esta organización facilita localizar cada tipo de información y evita mezclar archivos de desarrollo con archivos generados durante la ejecución.

Para la entrega académica también se incluye una carpeta destinada a documentación técnica y diagramas UML.

25. RESPALDO Y RECUPERACIÓN

La persistencia local permite conservar las órdenes después de cerrar la aplicación.

Los archivos Excel mensuales funcionan además como mecanismo complementario de respaldo y consulta.

En caso de requerir recuperación de información, los archivos JSON permiten reconstruir el estado utilizado por la aplicación, mientras que los documentos históricos conservan información legible para revisión administrativa.

Se recomienda realizar periódicamente una copia de seguridad de la carpeta correspondiente a datos y registros mensuales.

En una futura versión empresarial, este procedimiento podría sustituirse por respaldos automáticos sobre un sistema de base de datos.

26. MANTENIMIENTO

La estructura del código fue diseñada para permitir modificaciones futuras sin concentrar toda la lógica dentro de un único archivo.

La separación entre modelos, servicios, interfaz y repositorios permite identificar con mayor facilidad la ubicación de cada comportamiento.

Por ejemplo, una modificación relacionada con almacenamiento puede realizarse dentro de los componentes encargados de persistencia sin necesidad de modificar la manera en que la consola presenta las opciones al operador.

De manera similar, las reglas relacionadas con órdenes de servicio pueden mantenerse dentro de la capa correspondiente sin depender completamente del almacenamiento utilizado.

Las pruebas automatizadas constituyen también una herramienta para mantenimiento, ya que permiten comprobar que modificaciones futuras no alteren comportamientos previamente establecidos.

27. LIMITACIONES ACTUALES

KASA Service Tracker fue desarrollado deliberadamente como un sistema compacto.

Actualmente la aplicación utiliza una interfaz basada en consola, lo que resulta adecuado para demostrar funcionamiento y arquitectura, aunque no representa la experiencia que probablemente utilizaría un producto empresarial definitivo.

El almacenamiento principal se realiza mediante archivos locales, lo que limita escenarios donde múltiples computadoras requieran acceder simultáneamente a las mismas órdenes.

Tampoco existen usuarios, roles ni mecanismos de autenticación.

Estas limitaciones son consecuencia directa del alcance del proyecto y no representan funcionalidades incompletas.

Una evolución futura podría incorporar una interfaz gráfica mediante tecnologías .NET, un sistema gestor de bases de datos y mecanismos de autenticación.

28. RESULTADOS DEL PROYECTO

El resultado obtenido es una aplicación funcional capaz de administrar el ciclo principal de una orden de servicio automotriz desde su registro hasta su terminación.

El proyecto demuestra el uso de C# para construir una solución organizada mediante programación orientada a objetos, separación de responsabilidades y persistencia de información.

También permite aplicar conceptos relacionados con pruebas unitarias, validación, manejo de excepciones, estructuras de datos, consultas mediante LINQ y organización de un flujo de trabajo de desarrollo.

La incorporación de archivos históricos, seguimiento de estados y tickets permite que la aplicación trascienda el funcionamiento de un CRUD básico y represente un proceso definido con inicio, seguimiento y cierre.

29. CONCLUSIONES

El desarrollo de KASA Service Tracker permitió implementar una solución compacta para un problema específico de seguimiento de órdenes automotrices.

Uno de los principales aprendizajes obtenidos durante el proyecto fue la importancia de controlar el alcance. En lugar de intentar construir un sistema empresarial completo, se seleccionó un conjunto reducido de funcionalidades relacionadas entre sí que pudieron ser desarrolladas, probadas y organizadas correctamente.

La aplicación también permitió observar que una solución de software no depende únicamente de registrar información. Resulta igualmente importante considerar cómo será consultada, corregida, almacenada y recuperada con el paso del tiempo.

La decisión de utilizar un listado semanal junto con búsquedas específicas evita convertir la operación cotidiana en una visualización interminable de registros históricos.

Por otro lado, el uso combinado de persistencia JSON y exportación mensual permite separar información utilizada directamente por la aplicación de archivos destinados a consulta administrativa.

El proyecto permitió reforzar competencias relacionadas con C#, .NET, programación orientada a objetos, LINQ, archivos JSON, manejo de excepciones, pruebas con xUnit, diseño modular y control de versiones.

KASA Service Tracker representa de esta manera una evidencia práctica de desarrollo de software en la que se consideran tanto aspectos funcionales como técnicos y metodológicos.

30. COMPETENCIAS TÉCNICAS DESARROLLADAS

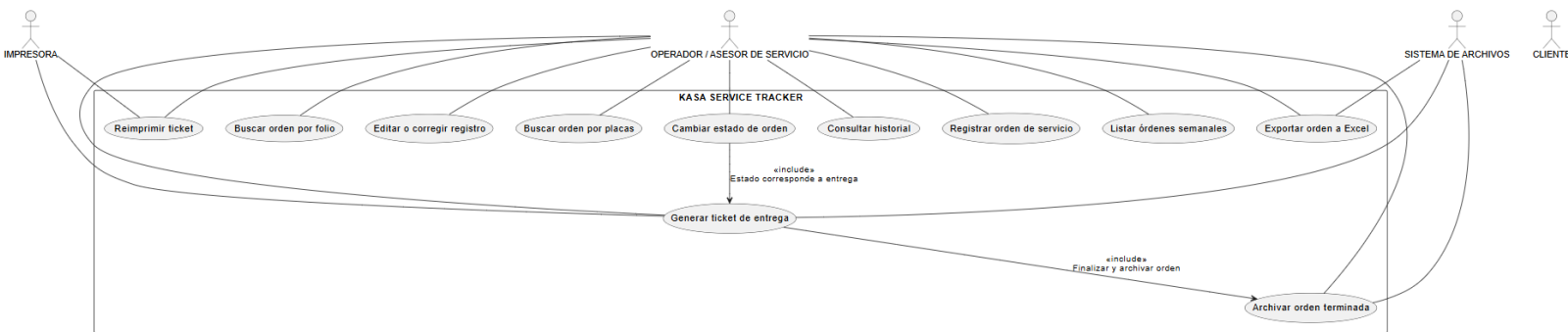
A partir del desarrollo de KASA Service Tracker fue posible aplicar conocimientos de programación con C# y .NET, programación orientada a objetos, clases, interfaces, enumeraciones, colecciones y LINQ.

También se trabajó con serialización y persistencia mediante JSON, manipulación y generación de archivos, validación de información, manejo de excepciones, pruebas unitarias mediante xUnit y separación de responsabilidades.

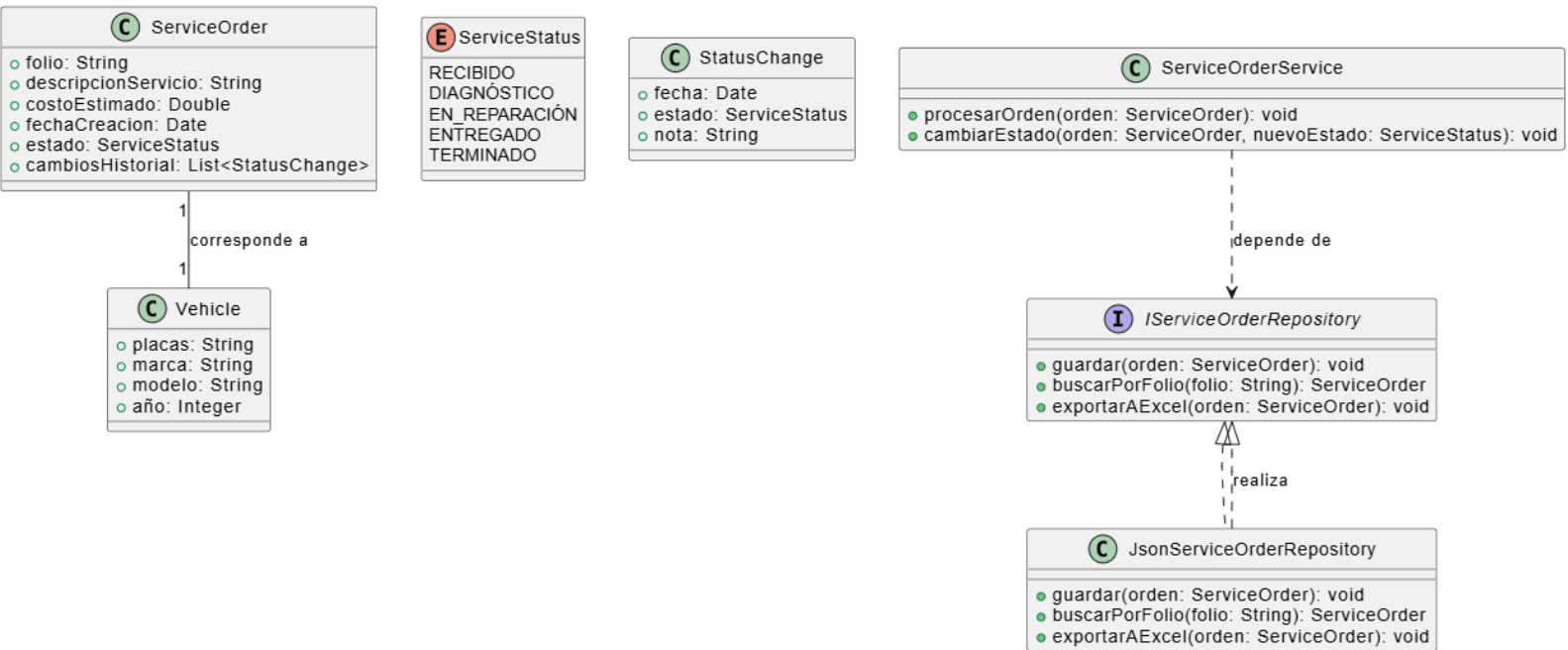
Desde la perspectiva de ingeniería de software, el proyecto permitió trabajar con análisis de requisitos, definición de alcance, arquitectura por capas, patrón Repository, trazabilidad, documentación técnica, UML, control de versiones y metodología Kanban.

31. UML

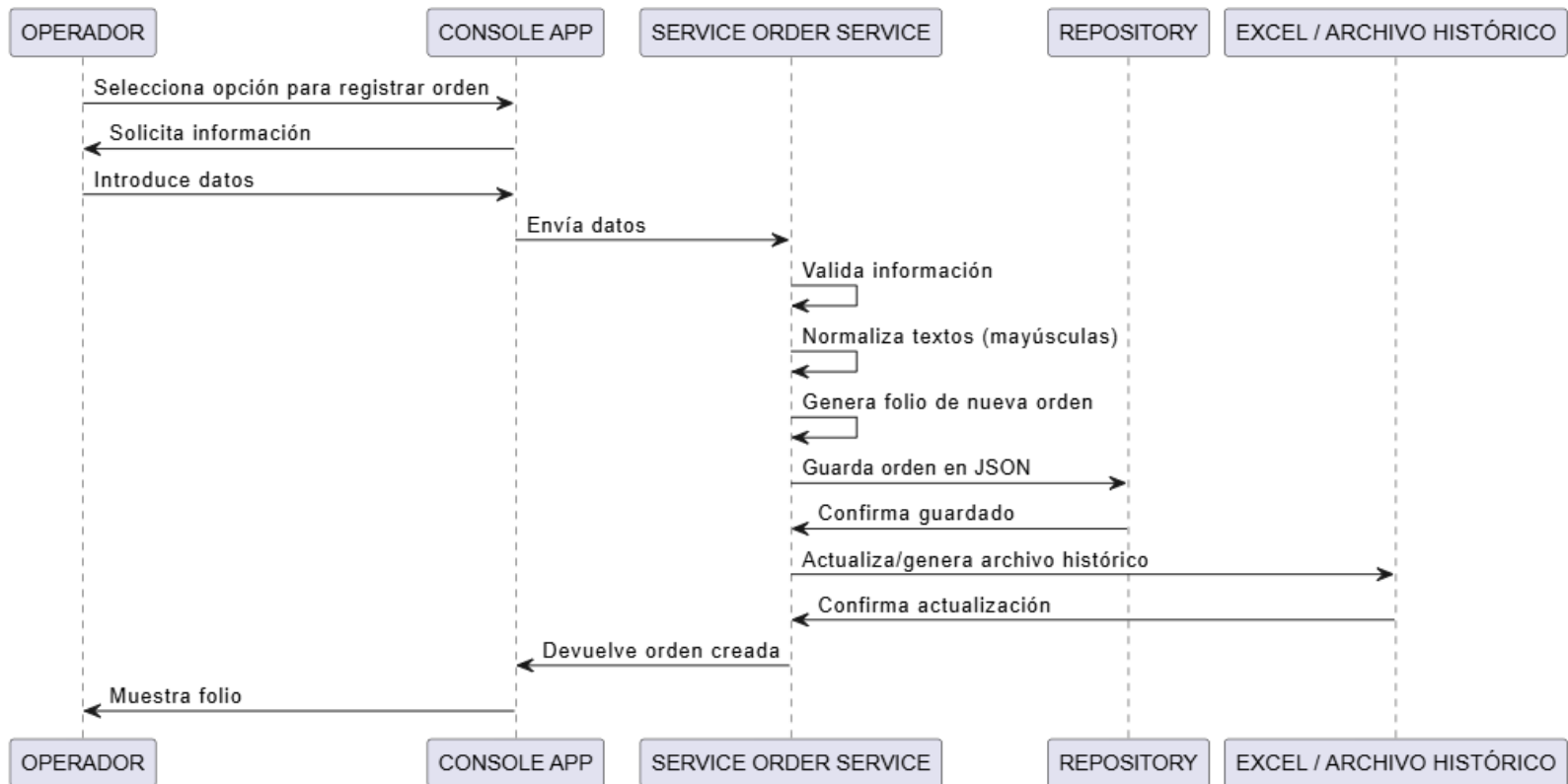
UML 1 — Diagrama de casos de uso



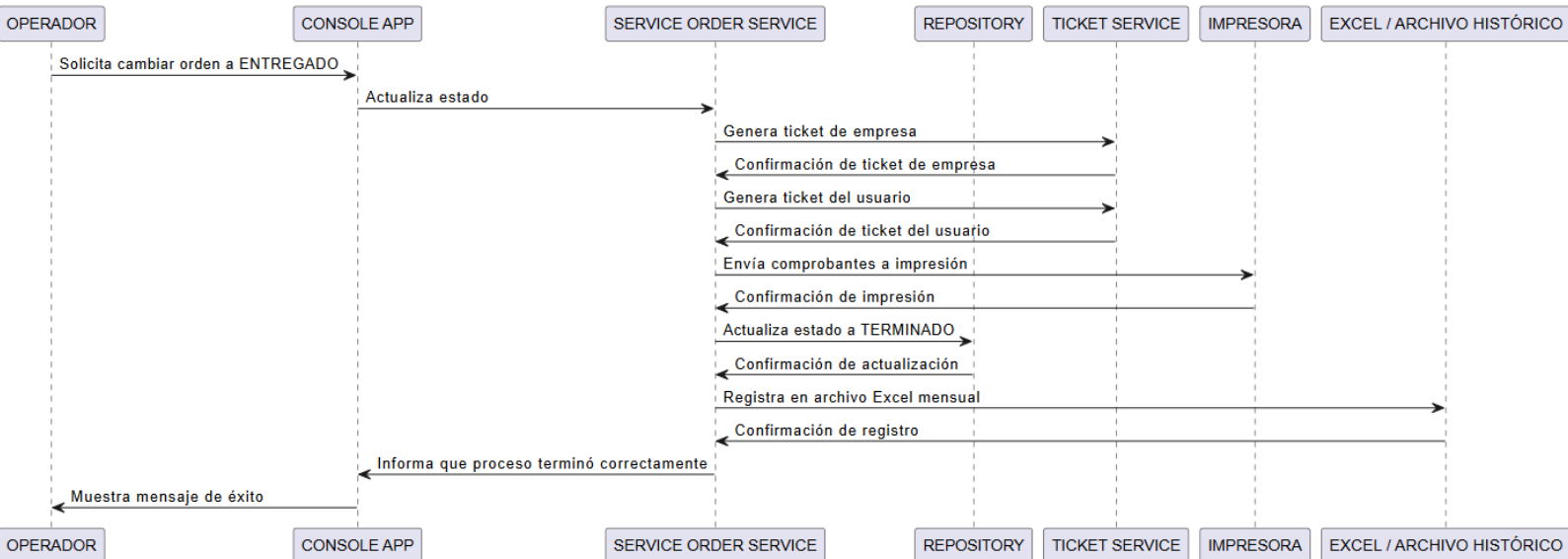
UML 2 — Diagrama de clases



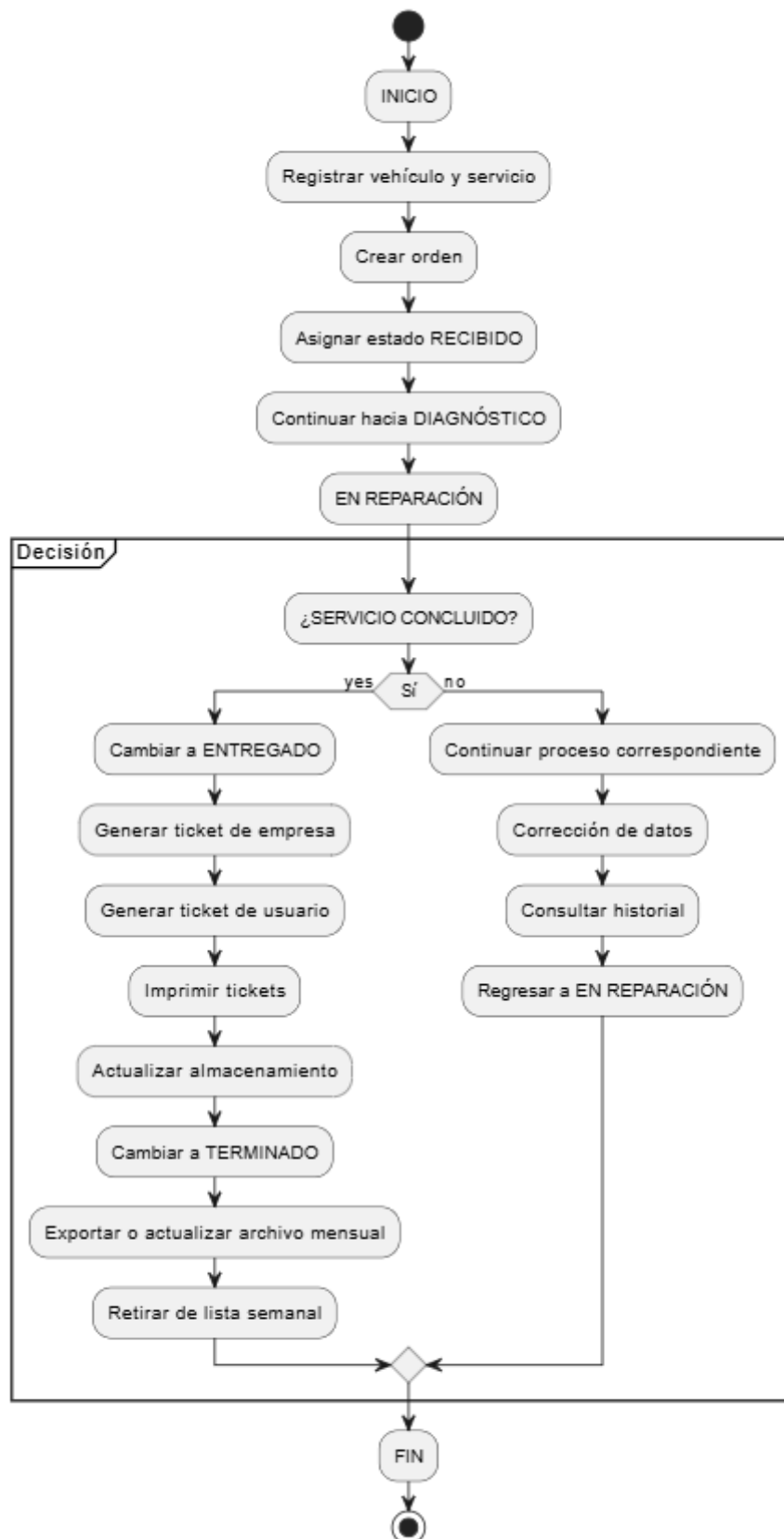
UML 3 — Diagrama de secuencia para registrar una orden



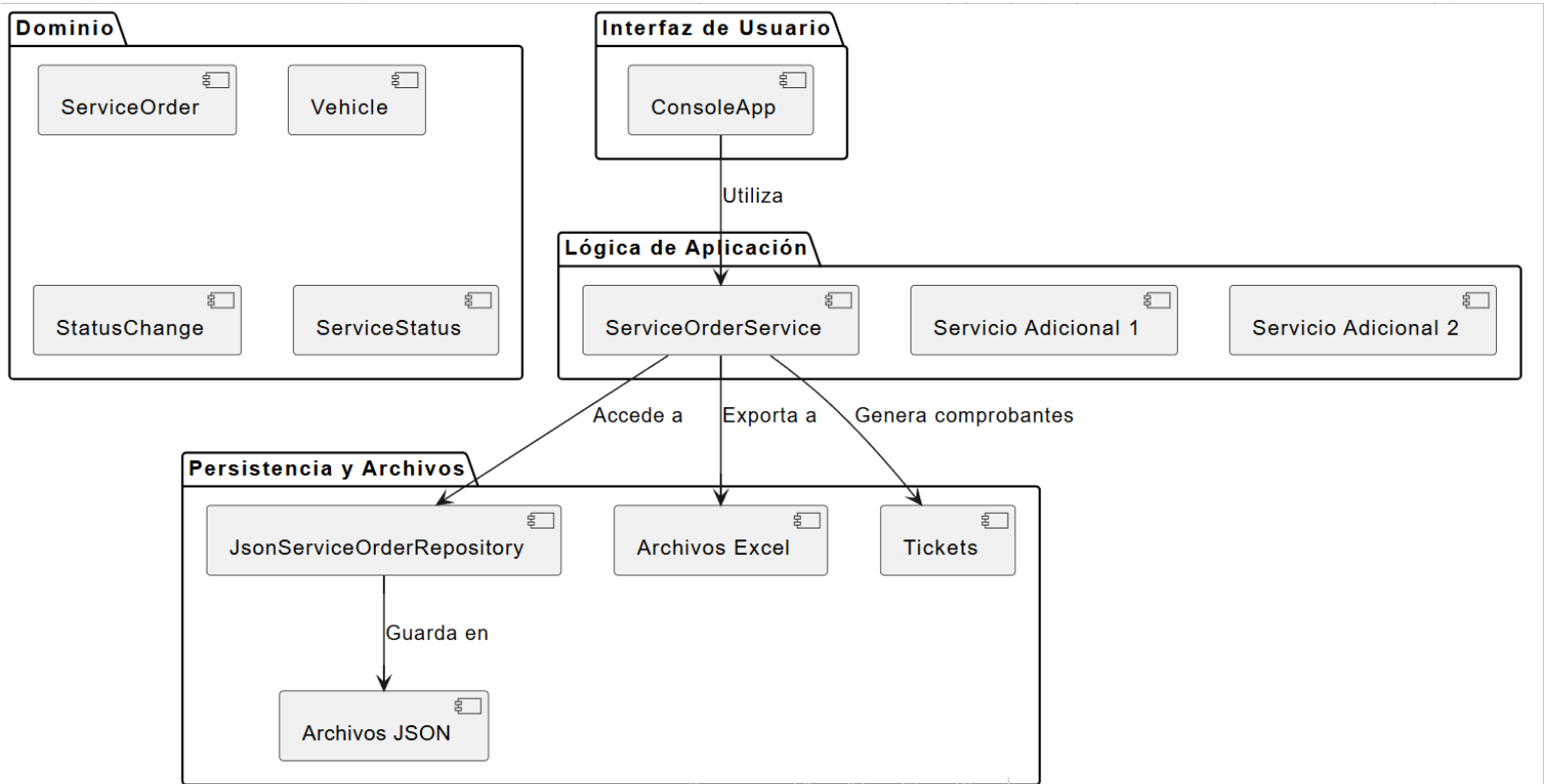
UML 4 — Diagrama de secuencia para entrega y finalización



UML 5 — Diagrama de actividad



UML 6 — Diagrama de componentes



UML 7 — Diagrama de despliegue

```

PlantUML version 1.2026.7beta11 / 1e6ec3d [2026-07-23 20:07:20 UTC]
[From string (line 16) ]

@startuml
node "Computadora Windows" {
    component "KASA Service Tracker"
    component ".NET Runtime"

    folder "Carpeta Data" {
    }
    folder "Carpeta Registros Excel" {
    }
    folder "Carpeta Tickets" {
    }
}

node "Impresora" as Impresora

[ESCRITORIO WINDOWS] --> [ACCESO DIRECTO] --> [KASA Service Tracker]
Syntax Error? (Assumed diagram type: component)
    
```

32. ANEXOS

<https://trello.com/invite/b/6a7cfabf83b0b0d1e1eaff36/ATTI7215d43c79c30476849eafb3fa52c88a8F583B10/kasa-service-tracker>

